Exemplar Questions for Class Test 1

Q1: Identify the Output

Question: What will be the output of the following code snippet?

```
public class Quiz1 {
    public static void main(String[] args) {
        int a = 10;
        int b = 4;
        double result = a / b;
        System.out.println(result);
    }
}
```

Answer:

```
2.0
```

- **Explanation:**
  Both `a` and `b` are integers. Integer division (`10 / 4`) gives `2`.
  This integer value is then ***promoted*** to `double` when assigned to `result`, producing `2.0`.

Q2: Correct the Code

Question: Find and correct the error in the following code:

```
public class Quiz2 {
    public static void main(String[] args) {
        int[] marks = new int[3];
        marks = {85, 90, 95};
        System.out.println(marks[2]);
    }
}
```

Answer (Corrected Code):

```
public class Quiz2 {
    public static void main(String[] args) {
        int[] marks = {85, 90, 95};
        System.out.println(marks[2]);
    }
}
```

Explanation:

- Array initializers (`{85, 90, 95}`) can only be used at the time of ***declaration***.

- Once an array is already created using `new`, you must assign elements individually (e.g., `marks[0] = 85;`).

Q3: Write a for-each loop

Question: Write a Java code snippet that prints all elements of the following array:

`int[] nums = {3, 6, 9, 12, 15};` using a ***for-each loop***.

Answer:

```
int[] nums = {3, 6, 9, 12, 15};
for (int n : nums) {
    System.out.print(n + " ");
}
```

Output:

```
3 6 9 12 15
```

Explanation:

- The variable `n` takes the value of each element in the array automatically—no index variable is needed.

Q4: Trace the Output of a 2D for-each Loop

Question: What will the following program print?

```
public class Quiz4 {
    public static void main(String[] args) {
        int[][] data = {
            {1, 2, 3},
            {4, 5},
            {6, 7, 8, 9}
        };

        for (int[] row : data) {
            for (int value : row) {
                System.out.print(value + " ");
            }
        }
    }
}
```

Answer:

```
1 2 3 4 5 6 7 8 9
```

Explanation:

- Each `row` is a 1D array (ragged).
- The outer loop iterates through each row, and the inner loop iterates through the row's elements.

Q5: Type Casting and Promotion

Question: What will be the output of the following code?

```
public class Quiz5 {
    public static void main(String[] args) {
        byte a = 40;
        byte b = 50;
        byte c = 100;
        int result = a * b / c;
        System.out.println(result);
    }
}
```

Answer:

```
20
```

Explanation:

- Before performing the arithmetic operation, all `byte` values are ***promoted to int***.
- So, `a * b / c` → `(40 * 50) / 100` → `2000 / 100` = `20`.

Q6: Object Creation and Reference Behavior

Task: Consider the following code:

```
class Box {
    double width, height, depth;
}

public class BoxDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = b1;
```

```
        b1.width = 5;
        b2.height = 10;
        System.out.println(b1.height);
    }
}
```

Question: What will be the output of the program? Explain briefly why.

Answer:
Output:

```
10.0
```

Explanation:

- `b1` and `b2` refer to the same `Box` object. Therefore, any changes made through `b2` are reflected in `b1`.

Q7: Constructor and Default Values

Task: Write a small Java class named `Student` with ***two instance variables***: `String name` and `int id`. Create a `main()` method that:

1. Creates an object using the ***default constructor***,
2. Prints the default values of both variables.

Answer:

```
class Student {
    String name;
    int id;
}

public class StudentDemo {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println("Name: " + s.name); // null
        System.out.println("ID: " + s.id);     // 0
    }
}
```

Explanation:

- Since no constructor is defined, Java provides a ***default constructor***, initializing `String` to `null` and `int` to `0`.

Q8: Fixing Instance Variable Hiding

Task: The following code has a bug due to ***variable hiding***. Correct it.

```
class Rectangle {
    int width, height;
    Rectangle(int width, int height) {
        width = width;
        height = height;
    }
}
```

Answer (Corrected Code):

```
class Rectangle {
    int width, height;
    Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
}
```

Explanation:

- Local variables (`width`, `height`) hide instance variables of the same name. Using `this.width` refers to the object's own instance variables.

Q9: Predicting Output with Parameterized Constructor

Task: Predict the output of the following code:

```
class MyClass {
    int x;
    MyClass(int i) {
        x = i;
    }
}

public class Test {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(5);
        MyClass obj2 = new MyClass(10);
        System.out.println(obj1.x + " " + obj2.x);
    }
}
```

Answer:
Output:

```
5 10
```

Explanation:

- Each object has its own copy of the instance variable `x`. The constructor initializes it with the value passed as an argument.

Q10: finalize() and Garbage Collection

Task:
What will be printed after running the following code? (Assume Garbage Collection runs.)

```
class TestGC {
    protected void finalize() {
        System.out.println("Object is being garbage collected");
    }

    public static void main(String[] args) {
        TestGC obj = new TestGC();
        obj = null;
        System.gc();  // Request garbage collection
    }
}
```

Answer:
Possible Output:

```
Object is being garbage collected
```

Explanation:

- When the object has no references (`obj = null`), it becomes eligible for garbage collection.
- The JVM may call `finalize()` before reclaiming memory.
- Note that calling `System.gc()` only **requests** GC — it's not guaranteed.